Linux Ext4 Internals: Architecture, Features, and Performance Tuning

Day 2: Journaling & Metadata Management

Duration & Level

Duration: 3.5 hours

Level: Expert

Instructor

Instructor: Chandrashekar Babu

Email: training@chandrashekar.info

Website:
https://www.chandrashekar.info/

https://www.slashprog.com/

Day 2 Agenda

What We'll Cover Today

Module 3: Journaling Mechanisms (1.5 hours)

- Journaling modes: ordered, writeback, journal
- Transaction lifecycle (begin, commit, checkpoint)
- Journal recovery and crash consistency
- Barrier semantics and write ordering
- Fsync and sync handling internals

Module 4: Metadata Operations (1.5 hours)

- Inode and block allocation algorithms
- Directory operations and HTree optimizations
- Extended attributes storage and management
- Quota internals and debugging
- Metadata recovery mechanisms

Hands-On Labs (30 minutes)

- Journal recovery simulation
- ftrace analysis of transactions
- Quota debugging with tracepoints

Break: 10 minutes between modules

Journaling Fundamentals

Why Journaling?

Problem: Crash Inconsistency

Without Journaling:

1. Create file (update directory)
2. Allocate inode (update bitmap)
3. Write data blocks

↓ ⚡ POWER LOSS ⚡ ↓

Result: Directory points to unallocated inode OR Inode points to wrong data OR Bitmap shows allocated but unused blocks

Solution: Transaction-Based Updates

With Journaling:

1. Begin transaction
2. Write all changes to journal (intent log)
3. Commit transaction (atomic)
4. Checkpoint changes to main filesystem
5. Free journal space

After crash: Replay completed transactions, discard incomplete ones

Three Consistency Models

1. Journal

Full data + metadata journaling

Slowest, Safest

2. Ordered

Metadata journaled, data ordered

Default, Balanced

3. Writeback

Metadata journaled only

Fastest, Riskier

JBD2 (Journaling Block Device 2)

The Journaling Layer

Architecture Overview

```
┌──────────────────────────┐
│ File System              │ ← ext4, ext3
│ Operations               │
└──────────────────────────┘
        ↓
┌──────────────────────────┐
│ JBD2                     │ ← Journaling layer
│ (journaling)             │   - Transactions
└──────────────────────────┘
        ↓                      - Handles
                               - Recovery
┌──────────────────────────┐
│ Block Device             │ ← Physical storage
└──────────────────────────┘
```

Key JBD2 Structures

```
struct journal_s {
 struct buffer_head *j_sb_buffer;
 journal_superblock_t *j_superblock;
 struct transaction_s *j_running_transaction;
 struct transaction_s *j_committing_transaction;
 struct transaction_s *j_checkpoint_transactions;
 wait_queue_head_t j_wait_transaction_locked;
 spinlock_t j_state_lock;
 unsigned long j_flags;
 int j_barrier_count;
 struct block_device *j_dev;
};
struct transaction_s {
 tid_t t_tid;
 enum {
 T_RUNNING, /* Accepting new handles */
 T_LOCKED, /* About to commit */
 T_FLUSH, /* Flushing buffers */
 T_COMMIT, /* Committing */
 T_COMMIT_DFLUSH, /* Data flush */
 T_COMMIT_JFLUSH, /* Journal flush */
 T_COMMIT_CALLBACK,/* Running callbacks */
 T_FINISHED /* Complete */
 } t_state;
 struct journal_head *t_buffers;
 struct journal_head *t_sync_datalist;
 struct journal_head *t_log_list;
};
```

Journaling Modes — Deep Dive

Mode Comparison

1. data=ordered (Default)

Write Flow:

1. Write data blocks to main FS (but don't update metadata)
2. Journal metadata changes
3. Commit transaction
4. Update metadata pointers to new data

Recovery: Data always consistent with metadata. Data may be stale if crash before commit.

2. data=writeback

Write Flow:

1. Write data blocks (anytime, anywhere)
2. Journal metadata changes
3. Commit transaction

Recovery: Metadata consistent, data may be garbage. Data written after metadata commit may be lost.

3. data=journal

Write Flow:

1. Write data to journal first
2. Write metadata to journal
3. Commit transaction
4. Checkpoint data to main FS
5. Checkpoint metadata to main FS

Recovery: Complete consistency. Data always recoverable.

Performance Characteristics

Mode	Write Throughput	fsync Latency	Data Safety
writeback	Highest	Lowest	Low
ordered	Medium	Medium	Medium
journal	Lowest	Highest	High

Transaction Lifecycle — Begin

Phase 1: Transaction Start

Starting a Transaction

```
// Code flow: ext4_journal_start()
handle_t *ext4_journal_start_sb(struct super_block *sb, int nblocks) {
    journal_t *journal = EXT4_SB(sb)->s_journal;

    if (!journal)
        return ERR_PTR(-EROFS);

    // Get or create transaction
    return jbd2__journal_start(journal, nblocks, GFP_NOFS);
}

static handle_t *jbd2_journal_start(journal_t *journal, int nblocks) {
    handle_t *handle;

    // Allocate handle structure
    handle = journal_alloc_handle(journal);
    if (!handle)
        return ERR_PTR(-ENOMEM);

    // Start a new transaction or join existing
    start_this_handle(journal, handle, gfp_mask);

    return handle;
}

static int start_this_handle(journal_t *journal, handle_t *handle) {
    transaction_t *transaction;

    // Get running transaction or create new
    transaction = journal->j_running_transaction;
    if (!transaction) {
        transaction = jbd2_journal_new_transaction(journal);
        journal->j_running_transaction = transaction;
    }

    // Reserve space in journal
    handle->h_transaction = transaction;
    transaction->t_outstanding_credits += handle->h_buffer_credits;

    return 0;
}
```

Handle States

- **h_start**: When handle created
- **h_buffer_credits**: Reserved journal space
- **h_sync**: fsync requested
- **h_aborted**: Transaction aborted

Transaction Lifecycle — Commit

Phase 2: Transaction Commit

Commit Process

```
// Triggered by journal_stop() or timer
int jbd2_journal_stop(handle_t *handle) {
    transaction_t *transaction = handle->h_transaction;
    journal_t *journal = transaction->t_journal;

    // Decrement handle count
    if (--handle->h_ref > 0)
        return 0;

    // Last handle? Start commit
    if (atomic_dec_and_test(&transaction->t_updates)) {
        wake_up(&journal->j_wait_updates);

        if (journal->j_committing_transaction != transaction) {
            // Schedule commit
            jbd2_journal_commit_transaction(journal);
        }
    }
}

void jbd2_journal_commit_transaction(journal_t *journal) {
    transaction_t *transaction;

    // 1. Lock transaction (no new handles)
    transaction = journal->j_running_transaction;
    transaction->t_state = T_LOCKED;

    // 2. Write descriptor block
    journal_write_descriptor_block(transaction);

    // 3. Write data blocks to journal (if needed)
    if (transaction->t_sync_datalist)
        journal_write_data_to_journal(transaction);

    // 4. Write commit block
    journal_write_commit_record(transaction);

    // 5. Wait for I/O completion
    journal_wait_for_io(transaction);

    // 6. Move to checkpointing
    transaction->t_state = T_COMMIT;
    journal->j_committing_transaction = NULL;
    journal->j_checkpoint_transactions = transaction;
}
```

Transaction Lifecycle — Checkpoint

Phase 3: Checkpointing

Checkpoint Process

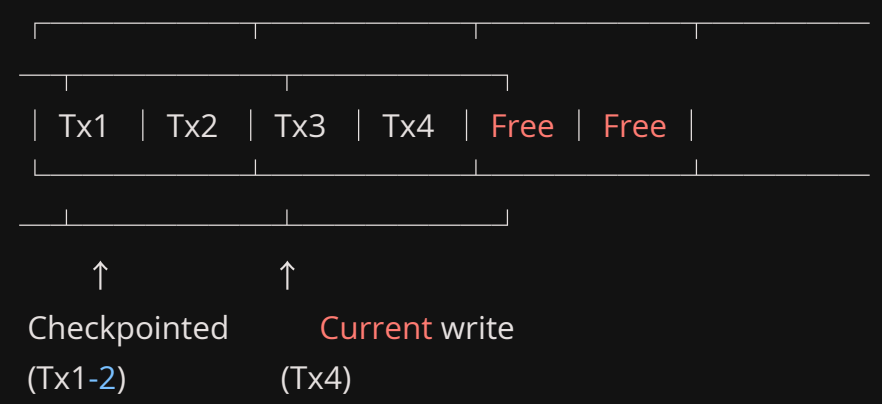
```
// Write journaled blocks to main filesystem
int jbd2_journal_destroy_checkpoint(journal_t *journal) {
    transaction_t *transaction;

    while ((transaction =
            journal->j_checkpoint_transactions)) {
        // For each buffer in transaction
        while (buffer = transaction->t_buffers) {
            // Write buffer to main FS
            if (buffer_dirty(buffer)) {
                write_buffer(buffer);
            }

            // Remove from journal
            journal_remove_journal_head(buffer);

            // Update journal tail
            transaction->t_checkpoint_io_done++;
        }

        // Transaction fully checkpointed
        if (all_buffers_written(transaction)) {
            journal->j_checkpoint_transactions = transaction->t_cpnext;
            jbd2_journal_free_transaction(transaction);
            journal->j_tail = transaction->t_tid;
        }
    }
}
```

Checkpoint Triggers

1. **Timer-based**: Periodic (commit=5s default)
2. **Space-based**: Journal 75% full
3. **Memory pressure**: Shrinker callback
4. **Sync request**: sync() or fsync()
5. **Umount**: Write all transactions

Journal Space Management

Journal Layout



Journal Recovery

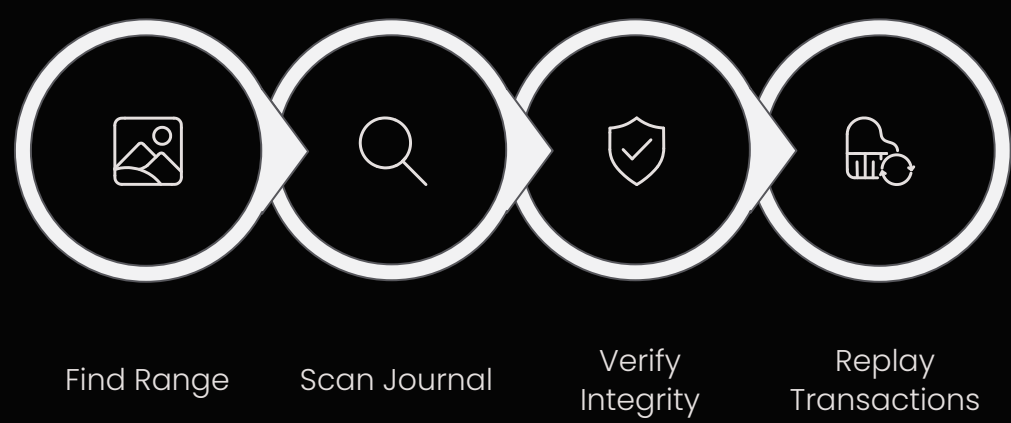
Recovery Algorithm

Recovery Flow

```
int jbd2_journal_recover(journal_t *journal) {
    journal_superblock_t *sb =
            journal->j_superblock;
    tid_t start, end;

    // 1. Find recovery range
    start = sb->s_first;
    end = sb->s_last;

    // 2. Scan journal from last checkpoint
    for (tid = start; tid <= end; tid++) {
        // Read transaction
        transaction = journal_read_transaction(journal, tid);

        // 3. Verify transaction integrity
        if (!transaction_valid(transaction))
            continue;

        // 4. Check commit block
        if (!commit_block_valid(transaction))
            continue;

        // 5. Replay blocks
        for_each_block_in_transaction(transaction, block) {
            // Write block to main filesystem
            ext4_replay_block(journal->j_fs_dev, block);
        }

        // 6. Update journal head
        journal->j_head = tid + 1;
    }

    return 0;
}
```

Validation Checks

```
static int transaction_valid(transaction_t *trans) {
  // Check descriptor block magic
  if (trans->descriptor->magic !=
      JBD2_MAGIC_NUMBER)
    return 0;
  // Verify checksum
  if (trans->descriptor->checksum !=
      calculate_checksum(trans))
    return 0;
  // Check transaction ID sequence
  if (trans->tid != expected_tid)
    return 0;
  return 1;
}
```



Barrier Semantics

Write Barriers Deep Dive

The Problem: Write Reordering

Without Barrier (Disk Cache Enabled):

1. Journal commit block written
2. *Disk reorders* — Metadata written after commit
3. Crash occurs
4. Journal says committed, but metadata lost

With Barrier:

1. Issue barrier before commit block
2. All previous writes flushed
3. Write commit block
4. Issue barrier after commit
5. Order guaranteed

Barrier Implementation

```
// In ext4 code
static void ext4_journal_commit_callback(
    journal_t *journal) {
  if (test_opt(sb, BARRIER)) {
    blkdev_issue_flush(sb->s_bdev, GFP_NOFS);
  }
}

int blkdev_issue_flush(struct block_device *bdev,
                       gfp_t gfp_mask) {
  struct request_queue *q = bdev_get_queue(bdev);
  if (!q->make_request_fn) return -ENXIO;
  bio = bio_alloc(gfp_mask, 0);
  bio->bi_opf = REQ_OP_FLUSH | REQ_PREFLUSH;
  bio->bi_bdev = bdev;
  submit_bio_wait(bio);
  bio_put(bio);
  return 0;
}
```

Performance Impact

Configuration	Throughput	Safety	Use Case
barrier=1	Baseline	Highest	Default, critical data
barrier=0	+10-40%	Low	Battery-backed RAID, testing
nobarrier + cache=writeback	+50-100%	None	Benchmarks only

Fsync and Sync Handling

Fsync Internals

ext4_sync_file() Code Path

```
// fsync system call handler
int ext4_sync_file(struct file *file, loff_t start, loff_t end, int datasync) {
    struct inode *inode = file->f_mapping->host;
    journal_t *journal = EXT4_SB(inode->i_sb)->s_journal;
    int ret, err;
    tid_t commit_tid;

    // 1. Write dirty pages
    ret = file_write_and_wait_range(file, start, end);
    if (ret)
        return ret;

    // 2. Get current transaction ID
    commit_tid = datasync ? ei->i_datasync_tid : ei->i_sync_tid;

    // 3. Force journal commit
    if (journal) {
        ret = jbd2_complete_transaction(journal, commit_tid);
        if (ret)
            return ret;
    }

    // 4. Flush disk cache if needed
    if (test_opt(inode->i_sb, BARRIER)) {
        err = blkdev_issue_flush(inode->i_sb->s_bdev, GFP_KERNEL);
        if (!ret)
            ret = err;
    }

    return ret;
}

// Transaction completion
int jbd2_complete_transaction(journal_t *journal, tid_t tid) {
    // Wait for transaction to commit
    wait_event(journal->j_wait_done_commit,
               tid_geq(journal->j_commit_sequence, tid));

    // Wait for checkpoint if needed
    if (tid_geq(journal->j_checkpoint_sequence, tid))
        return 0;

    // Force checkpoint
    return jbd2_log_wait_commit(journal, tid);
}
```

Fsync Optimisations

Reducing Fsync Overhead

1. Group Commit

```
// Multiple fsyncs coalesced
Time │
     ├──── fsync A ────┐
     ├──── fsync B ────┼──── Transaction Commit
     ├──── fsync C ────┘
     │
     ▼
```

2. Selective Flushing

```
// Only flush changed inodes
static int ext4_sync_parent(struct inode *inode) {
  struct dentry *dentry = NULL;
  while (inode && (S_ISDIR(inode->i_mode) ||
    !(dentry = d_find_any_alias(inode)))) {
    if (dentry &&
        dentry->d_parent != dentry) {
      inode = dentry->d_parent->d_inode;
      ext4_sync_inode(inode, 1);
    }
  }
}
```

3. Barrier Elimination

- Battery-backed RAID: disable barriers safely
- NVMe with power-loss protection: reduced flushing

Performance Numbers

Operation	With Fsync	Without Fsync	Group Commit
1x 4KB write	200μs	20μs	200μs
100x 4KB writes	20ms	2ms	2ms
1000x 4KB writes	200ms	20ms	10ms

Debugging Journal Issues

Journal Tracing with ftrace

Setup Journal Tracing

```
# Enable JBD2 tracepoints
echo 1 > /sys/kernel/debug/tracing/events/jbd2/enable

# Available tracepoints
ls /sys/kernel/debug/tracing/events/jbd2/
# jbd2_checkpoint       jbd2_commit_flushing
# jbd2_commit_logging   jbd2_commit_locking
# jbd2_end_commit       jbd2_run_stats
# jbd2_start_commit     jbd2_submit_inode_data

# Trace specific operations
echo 'jbd2_start_commit' > \
  /sys/kernel/debug/tracing/set_event
echo 'jbd2_end_commit' >> \
  /sys/kernel/debug/tracing/set_event

# Run workload
dd if=/dev/zero of=testfile bs=4k count=1000 conv=fsync

# View trace
cat /sys/kernel/debug/tracing/trace
```

Sample Output

```
dd-1234 [001] ...1 123.456: jbd2_start_commit:
  dev 8:1 transaction 1234
dd-1234 [001] ...1 123.457: jbd2_commit_locking:
  dev 8:1 transaction 1234
dd-1234 [001] ...1 123.458: jbd2_commit_flushing:
  dev 8:1 transaction 1234
jbd2/dm-0-8 [002] ...1 123.460: jbd2_end_commit:
  dev 8:1 transaction 1234
```

Debugging with perf

```
# JBD2 performance stats
perf stat -e jbd2:* -a -- sleep 10

# Call graph analysis
perf record -e jbd2:* -ag -- sleep 10
perf report
```

Journal Recovery Simulation

Lab: Crash Simulation

Setup Test Environment

```
# Create test filesystem
dd if=/dev/zero of=/tmp/ext4_test.img \
  bs=1M count=1024
mkfs.ext4 -F /tmp/ext4_test.img

# Mount with journal
mkdir /mnt/ext4_test
mount -o loop,data=ordered \
 /tmp/ext4_test.img /mnt/ext4_test

# Create workload
cd /mnt/ext4_test
for i in {1..1000}; do
 echo "Data $i" > file$i
done
```

Simulate Crash

```
# Method 1: Force power loss (VM environment)
echo 1 > /proc/sys/kernel/sysrq
echo b > /proc/sysrq-trigger

# Method 2: Emergency remount
mount -o remount,ro /mnt/ext4_test
# Then unplug (in VM)

# Method 3: Use fault injection
echo Y > /sys/module/jbd2/parameters/jbd2_debug
# Crash during journal commit
```

Recovery Analysis

```
# After reboot/crash
mount /dev/loop0 /mnt/ext4_test
dmesg | grep -i journal

# Check filesystem
umount /mnt/ext4_test
e2fsck -f /tmp/ext4_test.img

# Examine journal
debugfs -R "logdump -a" /tmp/ext4_test.img
```

Module 3 Summary

Journaling Key Takeaways

Transaction Lifecycle

1. **BEGIN**: Create handle, join/start transaction
2. **COMMIT**: Write descriptor, data, commit block
3. **CHECKPOINT**: Write to main FS, free journal space

Journaling Modes

- **ordered** (default): Best balance for most workloads
- **writeback**: Maximum performance, some data risk
- **journal**: Maximum safety, significant overhead

Barrier Semantics

- **barrier=1**: Guaranteed ordering, safe
- **barrier=0**: Faster, risk on power loss
- Hardware matters: NVMe vs HDD vs SSD cache

Fsync Implementation

- Forces journal commit
- Waits for I/O completion
- Optional cache flush
- Group commit optimisation

Recovery Process

- Scan journal from checkpoint
- Verify transaction integrity
- Replay committed transactions
- Skip incomplete ones

Module 4 — Metadata Operations

Introduction to Metadata Management

Types of Metadata

```
Filesystem Metadata:
├── Superblock (global FS info)
├── Group descriptors (per-group info)
├── Block bitmaps (free/allocated blocks)
├── Inode bitmaps (free/allocated inodes)
├── Inode tables (file metadata)
├── Directory entries (name→inode mapping)
├── Extent trees (block mapping)
└── Extended attributes (xattr)
```

Metadata Operations

- **Allocation**: Finding free blocks/inodes
- **Deallocation**: Reclaiming space
- **Modification**: Updates to structures
- **Validation**: Ensuring consistency
- **Journaling**: Transaction logging

Performance Considerations

- Metadata operations are small random I/O
- Caching critical for performance
- Group locality reduces seeks
- Checksums add CPU overhead

Inode Allocation Algorithm

Finding Free Inodes

Inode Allocation Path

```
struct inode *ext4_new_inode(handle_t *handle,
    struct inode *dir, ...) {
  struct super_block *sb = dir->i_sb;
  struct ext4_sb_info *sbi = EXT4_SB(sb);
  struct ext4_group_info *grp;
  int group = 0;

  // 1. Select target group
  if (S_ISDIR(mode)) {
    // Directories: spread across groups
    group = ext4_find_group_dir(sb, dir);
  } else {
    // Files: try to allocate near parent
    group = ext4_find_group_near(sb, dir);
  }

  // 2. Search within group
  ino = ext4_find_next_zero_bit(
    grp->inode_bitmap,
    sbi->s_inodes_per_group,
    grp->last_alloc);

  // 3. Try other groups if not found
  if (ino >= sbi->s_inodes_per_group) {
    group = ext4_find_group_other(sb, dir);
  }

  // 4. Reserve inode in journal
  BUFFER_TRACE(bitmap_bh, "journal_dirty_metadata");
  ext4_journal_dirty_metadata(handle, bitmap_bh);

  // 5. Initialize inode
  ext4_init_inode(inode, dir, mode);
  return inode;
}
```

Key Allocation Principles

Directories

Spread across groups using Orlov allocator for even distribution

Files

Allocated near parent directory to improve locality

Fallback

Search other groups when preferred group is full

Allocation Policies

Group Selection Strategies

Near Allocation (ext4_find_group_near)

```
static int ext4_find_group_near(
    struct super_block *sb,
    struct inode *parent) {
  struct ext4_sb_info *sbi = EXT4_SB(sb);
  struct ext4_inode_info *ei = EXT4_I(parent);

  // Try parent's group first
  parent_group = (ei->i_block_group);

  // Check if group has free inodes
  if (ext4_free_inodes_count(sb, gdp) &&
      ext4_free_blks_count(sb, gdp))
    return parent_group;

  // Fall back to other groups
  return ext4_find_group_other(sb, parent);
}
```

Orlov Allocation (for directories)

```
static int ext4_find_group_dir(
 struct super_block *sb,
 struct inode *parent) {
 // Orlov allocator: spread directories evenly
 // but keep related files together

 // Calculate group with most free space
 free_blocks = ext4_count_free_blocks(sb);
 avefreeb = free_blocks / ngroups;

 // Find group meeting criteria
 for (group = 0; group < ngroups; group++) {
 free_in_group =
 ext4_free_blks_count(sb, gdp);
 if (free_in_group >= avefreeb) {
 // Group has above-average free space
 break;
 }
 }
 return group;
}
```

Block Allocation — Multiblock Allocator (mballoc)

mballoc Architecture

Overview

```
mballoc replaces per-block allocation
with extent-based allocation
        ↓
 ┌──────────────────────────────────────
┐
 │ mballoc          │
 │ - Preallocation  │
 │ - Locality       │
 │ - Extent merging │
 └──────────────────────────────────────
┘
        ↓
 Block Bitmap Updates
```

Allocation Path

```
static int ext4_ext_map_blocks(handle_t *handle,
                               struct inode *inode,
                               struct ext4_map_blocks *map,
                               int flags) {
    // 1. Check existing extents
    ext4_ext_find_extent(inode, map->m_lblk, &path);

    if (found_extent) {
        // Use existing extent
        map->m_pblk = extent_start + (map->m_lblk - extent_block);
        map->m_len = min(map->m_len, extent_len);
    } else {
        // 2. Allocate new blocks
        ar.inode = inode;
        ar.goal = ext4_ext_find_goal(inode, path, map->m_lblk);
        ar.len = map->m_len;

        // mballoc allocation
        allocated = ext4_mb_new_blocks(handle, &ar, &err);

        // 3. Insert into extent tree
        ext4_ext_insert_extent(handle, inode, &path, &newext, flags);
    }
}
```

mballoc Preallocation Strategies

Preallocation Methods

1

Inode Preallocation (per-inode)

Single file allocation:

```
┌──────────────────────────────────────┐
│ File data blocks                     │
├──────────────────────────────────────┤
│ Preallocated space                   │ ← Reserved for future writes
└──────────────────────────────────────┘
```

Benefit: Contiguous growth for files

2

Locality Group Preallocation (per-CPU)

Multiple small files in same directory:

```
┌──────────────────────────────────────────────────┐
│ File A blocks │ File B blocks │ Free             │
├──────────────────────────────────────────────────┤
│ Prealloc region (shared among files)             │
└──────────────────────────────────────────────────┘
```

Benefit: Small files share preallocated space

3

Allocation Criteria

```
struct ext4_allocation_request {
    struct inode *inode;     /* Target inode */
    ext4_lblk_t logical;     /* Logical block */
    ext4_lblk_t lleft;       /* Left logical neighbor */
    ext4_lblk_t lright;      /* Right logical neighbor */
    ext4_fsblk_t goal;       /* Physical goal */
    ext4_fsblk_t pleft;      /* Left physical neighbor */
    ext4_fsblk_t pright;     /* Right physical neighbor */
    unsigned int len;        /* Requested length */
    unsigned int flags;      /* Allocation flags */
};
```

Delayed Allocation

How Delalloc Works

Traditional vs Delayed Allocation

```
Traditional (ext3):
write() → allocate blocks → copy data → writeback
(Allocation at write time)

Delayed Allocation (ext4):
write() → reserve space → copy data
      → allocate at writeback
(Allocation delayed until necessary)
```

Code Flow

```
// Write path with delalloc
ssize_t ext4_write_begin(...) {
 // Reserve blocks but don't allocate
 ext4_da_reserve_space(inode, len);
 // Grab page for writing
 page = grab_cache_page_write_begin(
 mapping, index);
 // Reserve journal space
 handle = ext4_journal_start(inode,
 needed_blocks);
}

static int ext4_da_reserve_space(
 struct inode *inode,
 ext4_lblk_t lblock) {
 struct ext4_sb_info *sbi = EXT4_SB(inode->i_sb);
 // Calculate needed blocks
 nrblocks = ext4_es_scan_clu(inode, &lblock, 1);
 // Reserve in quota
 dquot_reserve_block(inode, nrblocks);
 // Reserve in delalloc metadata
 percpu_counter_add(
 &sbi->s_dirtyclusters_counter, nrblocks);
 return 0;
}
```

Delalloc Benefits and Pitfalls

Advantages

1. **Better allocation decisions** — See full write pattern
2. **Reduced fragmentation** — Allocate contiguous extents
3. **Fewer journal commits** — Batch allocations
4. **Improved throughput** — Less CPU overhead

Risks

1. **ENOSPC on writeback** — Space reserved but not allocated
2. **Data loss window** — If crash between write and writeback
3. **Memory pressure** — Dirty pages accumulate

ENOSPC Handling

```
// Writeback path - may fail if insufficient space
static int ext4_da_writepages(
 struct address_space *mapping,
 struct writeback_control *wbc) {
 // Check available space
 needed = calculate_needed_blocks(mapping);
 available = ext4_has_free_blocks(sbi, needed);

 if (!available) {
 // Fall back to non-delalloc path
 return ext4_writepages(mapping, wbc);
 }

 // Allocate blocks now
 ret = ext4_ext_map_blocks(handle, inode,
 map, flags);
}
```

Fragmentation Handling

Causes and Solutions

Fragmentation Types

```
1. Internal fragmentation:
   Wasted space within blocks
   ┌──────┬──────┬──────┐
   │ 2KB  │ 2KB  │ 4KB  │ ← 8KB file in 4KB blocks
   └──────┴──────┴──────┘
   (2KB wasted)

2. External fragmentation:
   Non-contiguous extents
   File A: [0-10] [20-30] [40-50] ← scattered
   File B: [11-19] [31-39]        ← interspersed
```

Prevention Strategies

```
// mballoc tries to allocate contiguous
static int ext4_mb_good_group(
    struct ext4_allocation_context *ac,
    struct ext4_group_info *grp, int cr) {
  if (grp->bb_fragments >
      ac->ac_g_ex.fe_len / 2)
    return 0; // Too fragmented
  if (grp->bb_free < ac->ac_g_ex.fe_len)
    return 0;
  return 1;
}
```

Defragmentation

```
# Online defragmentation
e4defrag /path/to/file
e4defrag /mount/point # Defrag entire FS

# Check fragmentation level
filefrag -v /path/to/file

# Kernel support
echo 1 > /proc/sys/fs/ext4/extent_max_zeroout_kb
```

Directory Operations

Directory Entry Structure

On-disk Format

```
struct ext4_dir_entry_2 {
  __le32 inode;      /* Inode number */
  __le16 rec_len;    /* Directory entry length */
  __u8   name_len;   /* Name length */
  __u8   file_type;  /* File type */
  char   name[EXT4_NAME_LEN]; /* File name */
};

/* File types */
#define EXT4_FT_UNKNOWN  0
#define EXT4_FT_REG_FILE 1
#define EXT4_FT_DIR      2
#define EXT4_FT_CHRDEV   3
#define EXT4_FT_BLKDEV   4
#define EXT4_FT_FIFO     5
#define EXT4_FT_SOCK     6
#define EXT4_FT_SYMLINK  7
```

Directory Operations

```
// Lookup
struct dentry *ext4_lookup(struct inode *dir,
 struct dentry *dentry, unsigned int flags) {
 ino = ext4_inode_by_name(dir,
 &dentry->d_name);
 inode = ext4_iget(dir->i_sb, ino);
 return d_splice_alias(inode, dentry);
}

// Create
int ext4_create(struct inode *dir,
 struct dentry *dentry,
 umode_t mode, bool excl) {
 handle = ext4_journal_start(dir,
 EXT4_DATA_TRANS_BLOCKS(dir->i_sb) +
 EXT4_INDEX_EXTRA_TRANS_BLOCKS + 3);
 inode = ext4_new_inode_start_handle(
 handle, dir, mode,
 &dentry->d_name, 0, NULL);
 ext4_add_entry(handle, dentry, inode);
}
```

HTree Directory Indexing

HTree Structure

Index Creation

```
// When directory grows beyond threshold
static int ext4_dx_add_entry(handle_t *handle,
    struct dentry *dentry,
    struct inode *inode) {
  struct dx_frame frames[EXT4_HTREE_LEVEL];
  struct dx_entry *entries, *at;

  // 1. Compute hash of filename
  hash = dx_hack_hash(name, name_len, hinfo);

  // 2. Navigate tree to find insertion point
  frame = dx_probe(dentry, &hinfo, frames, 0);

  // 3. Insert in leaf block
  err = ext4_handle_dirty_dx_node(handle,
                                  inode, frame->bh);

  // 4. Split if necessary
  if (need_split) {
    ext4_dx_split(handle, inode,
                  frames, &hinfo);
  }
}
```

Hash Functions

```
// Default hash (legacy)
static unsigned int dx_hack_hash(
 const char *name, int len) {
 unsigned int hash = 0;
 while (len--)
 hash = (hash << 5) + hash + *name++;
 return hash;
}

// Half MD4 (more uniform)
static unsigned int half_md4_hash(
 const char *name, int len) {
 // More complex, better distribution
 return half_md4(name, len);
}

// TEA (encryption-based)
static unsigned int tea_hash(
 const char *name, int len) {
 // Tiny Encryption Algorithm
 return tea_encrypt(name, len);
}
```

HTree Operations

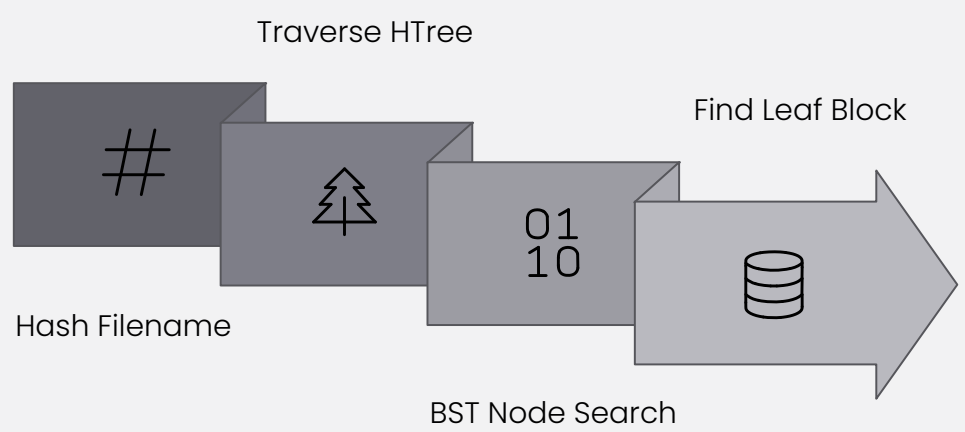
Lookup in HTree Directory

Algorithm

```
struct dentry *ext4_htree_lookup(
  struct inode *dir,
  struct dentry *dentry,
  struct dx_hash_info *hinfo) {
 struct dx_frame frames[EXT4_HTREE_LEVEL], *frame;
 struct dx_entry *entries;

 // 1. Compute filename hash
 hinfo->hash_version =
  EXT4_SB(dir->i_sb)->s_hash_unsigned;
 hinfo->seed = EXT4_SB(dir->i_sb)->s_hash_seed;
 ext4fs_dirhash(dentry->d_name.name,
            dentry->d_name.len, hinfo);

 // 2. Start at root
 frame = frames;
 frame->bh = ext4_get_first_dir_block(
  handle, dir, &retval, NULL);

 // 3. Navigate down the tree
 while (1) {
  entries = (struct dx_entry *)
          frame->bh->b_data;
  at = dx_bsearch_hash(hinfo->hash,
                entries, count);
  if (frame->bh == root) break;
  frame++;
  frame->bh = ext4_read_dirblock(dir,
            dx_get_block(at), 0);
 }

 // 4. Search leaf block
 de = ext4_search_dir(bh, hinfo->hash,
  dentry->d_name.name,
  dentry->d_name.len, &res_dir);
 return d_splice_alias(inode, dentry);
}
```

Performance Comparison

Directory Size	Linear Lookup	HTree Lookup
100 files	100 ops	~7 ops
10,000 files	10,000 ops	~14 ops
1M files	1M ops	~20 ops



Extended Attributes (xattr) Deep Dive

Storage Strategies

Inline xattr (within inode)

```
// xattr stored in inode's extra space
struct ext4_xattr_ibody_header {
  __le32 h_magic;    /* Magic (0xEA020000) */
  __le32 h_refcount; /* Reference count */
};

struct ext4_xattr_entry {
  __u8   e_name_len;    /* Length of name */
  __u8   e_name_index;  /* Attribute name index */
  __le16 e_value_offs;  /* Offset in value block */
  __le32 e_value_block; /* Block number (unused) */
  __le32 e_value_size;  /* Size of attribute value */
  __le32 e_hash;        /* Hash of name and value */
  char   e_name[0];     /* Attribute name */
};
```

External xattr Block

```
// When inline space exhausted
struct ext4_xattr_block_header {
 __le32 h_magic; /* Magic number */
 __le32 h_refcount; /* Reference count */
 __le32 h_blocks; /* Number of blocks (1) */
 __le32 h_hash; /* Hash of all attributes */
 __le32 h_checksum; /* Checksum */
};

// Allocation strategy
static int ext4_xattr_set(struct inode *inode,
 int name_index, const char *name,
 const void *value, size_t value_len,
 int flags) {
 // Try inline first
 if (ext4_xattr_can_set_in_inode(inode,
 &i, &bs))
 return ext4_xattr_set_in_inode(handle,
 inode, &i);
 // Fall back to external block
 return ext4_xattr_set_external(handle,
 inode, &i);
}
```

Quota Internals

Quota System Architecture

Quota Types

```
#define USRQUOTA 0  /* User quota */
#define GRPQUOTA 1  /* Group quota */
#define PRJQUOTA 2  /* Project quota */

struct dquot {
  struct list_head dq_hash;
  struct list_head dq_dirty;
  struct mutex     dq_lock;
  atomic_t         dq_count;
  wait_queue_head_t dq_wait_unused;
  struct super_block *dq_sb;
  unsigned int     dq_id;
  loff_t           dq_off;
  unsigned long    dq_flags;
  struct mem_dqblk dq_dqb;
};
```

Quota Limits Structure

```
struct mem_dqblk {
  qsize_t dqb_bhardlimit; /* Absolute hard limit */
  qsize_t dqb_bsoftlimit; /* Soft limit */
  qsize_t dqb_curspace; /* Current space used */
  qsize_t dqb_ihardlimit; /* Inode hard limit */
  qsize_t dqb_isoftlimit; /* Inode soft limit */
  qsize_t dqb_curinodes; /* Current inodes used */
  time64_t dqb_btime; /* Time for soft limit */
  time64_t dqb_itime; /* Inode soft limit time */
};
```

Quota Update Path

How Quota Updates Work

Allocation Time Update

```
int dquot_alloc_block(struct inode *inode,
                      qsize_t number) {
  struct dquot **dquots = i_dquot(inode);
  int cnt, ret = 0;

  // Update all quota types
  // (user, group, project)
  for (cnt = 0; cnt < MAXQUOTAS; cnt++) {
    if (dquots[cnt]) {
      ret = dquot_alloc_space(dquots[cnt],
                              number * 1024);
      if (ret) {
        // Rollback previous updates
        dquot_free_space(dquots[cnt],
                         number * 1024);
        break;
      }
    }
  }
  return ret;
}
```

Space Allocation Check

```
static int dquot_alloc_space(
 struct dquot *dquot, qsize_t number) {
 struct mem_dqblk *b = &dquot->dq_dqb;

 // Check hard limit
 if (b->dqb_bhardlimit &&
 (b->dqb_curspace + number) >
 b->dqb_bhardlimit) {
 return -EDQUOT;
 }

 // Update usage
 b->dqb_curspace += number;

 // Mark dirty for writeback
 mark_dquot_dirty(dquot);
 return 0;
}
```

Debugging Quota Errors

Quota Debugging Techniques

Using ftrace for Quota

```
# Enable quota tracepoints
echo 1 > \
  /sys/kernel/debug/tracing/events/quota/enable

# Available tracepoints
ls /sys/kernel/debug/tracing/events/quota/
# dquot_acquire       dquot_alloc_space
# dquot_free_space    dquot_release
# dquot_writeback_dquots

# Trace quota allocations
echo 'dquot_alloc_space' > \
  /sys/kernel/debug/tracing/set_event

# Run quota-intensive workload
dd if=/dev/zero of=/mnt/test/file bs=1M count=100

# Analyze trace
cat /sys/kernel/debug/tracing/trace
```

Sample Output

```
dd-1234 [001] ...1 123.456: dquot_alloc_space:
  dev 8:1 id=1000 type=USRQUOTA
  cur=0 new=1048576 limit=10485760
dd-1234 [001] ...1 123.457: dquot_alloc_space:
  dev 8:1 id=1000 type=USRQUOTA
  cur=1048576 new=2097152 limit=10485760
dd-1234 [001] ...1 123.458: quota_exceeded:
  dev 8:1 id=1000 type=USRQUOTA
  cur=11534336 limit=10485760
```

Debugging Tools

```
# Check quota usage
repquota -av
quota -v user

# Debug quota files
debugfs -R "stat quota.user" /dev/sda1

# Force quota sync
quotasync -av

# Quota check and repair
quotacheck -avugm
```

Metadata Recovery Mechanisms

Recovery Strategies for Metadata

Transaction-Based Recovery

```
// During journal replay
static int ext4_replay_blocks(
    journal_t *journal,
    transaction_t *transaction) {
  struct buffer_head *bh;

  // Replay each metadata block
  for (i = 0; i < transaction->t_nr_buffers;
       i++) {
    bh = transaction->t_buffers[i];

    // Verify block before replay
    if (!ext4_verify_metadata_block(bh)) {
      // Use backup if available
      bh = ext4_get_backup_block(
            bh->b_blocknr);
    }

    // Write to filesystem
    ext4_write_block(bh);
  }
}
```

Checksum Verification

```
// Metadata checksum calculation
__le32 ext4_inode_csum(struct inode *inode,
 struct ext4_inode *raw,
 struct ext4_super_block *es) {
 struct ext4_inode_tail *tail;
 __u32 csum;

 // Skip i_extra_isize when computing
 csum = ext4_chksum(sbi, sbi->s_csum_seed,
 (__u8 *)raw,
 offset + raw->i_extra_isize);

 // Include inode tail
 tail = (struct ext4_inode_tail *)
 (((void *)raw) +
 le16_to_cpu(raw->i_extra_isize));
 csum = ext4_chksum(sbi, csum,
 (__u8 *)tail,
 sizeof(struct ext4_inode_tail));

 return cpu_to_le32(csum);
}
```

Handling Metadata Corruption

Corruption Detection and Repair

Detection Path

```
// During inode read
struct inode *ext4_iget(struct super_block *sb,
                        unsigned long ino) {
  raw_inode = ext4_raw_inode(&iloc);

  // Verify inode checksum
  if (ext4_inode_csum_verify(sb, ino,
                             raw_inode, ei)) {
    // Checksum OK
  } else {
    // Checksum failed - corruption detected
    ext4_error(sb,
      "Inode %lu checksum mismatch", ino);

    // Try to recover from backup
    if (ext4_recover_inode(sb, ino,
                           raw_inode)) {
      // Recovery successful
    } else {
      return ERR_PTR(-EIO);
    }
  }
}
```

Recovery Attempt

```
static int ext4_recover_inode(
  struct super_block *sb,
  unsigned long ino,
  struct ext4_inode *raw) {
  struct ext4_group_desc *gdp;

  // Check backup group descriptors
  backup_group = ext4_find_backup_group(sb,
  ino);
  gdp = ext4_get_group_desc(sb,
  backup_group, NULL);

  // Read inode from backup
  backup_inode = ext4_read_inode_from_group(
  sb, ino, backup_group);

  // Compare and restore
  if (ext4_compare_inodes(raw,
  backup_inode) == 0) {
  memcpy(raw, backup_inode,
  EXT4_GOOD_OLD_INODE_SIZE);
  return 1;
  }
  return 0;
}
```

Hands-On Lab — Journal Mode Comparison

Lab Exercise: Benchmark Journal Modes

Setup

```
# Create test filesystem images
for mode in ordered writeback journal; do
  dd if=/dev/zero \
     of=/tmp/ext4_${mode}.img \
     bs=1M count=1024
  mkfs.ext4 -F /tmp/ext4_${mode}.img
  mkdir /mnt/${mode}
  mount -o loop,data=${mode} \
    /tmp/ext4_${mode}.img /mnt/${mode}
done
```

Fio Benchmark

```
# Create fio job file (journal-test.fio)
cat > journal-test.fio << EOF
[global]
ioengine=sync
bs=4k
size=256M
directory=/mnt/\$mode
stonewall

[seqwrite]
rw=write
name=seqwrite

[randwrite]
rw=randwrite
name=randwrite

[fsync-test]
rw=write
bs=4k
size=16M
fsync=1
name=fsync-test
EOF

# Run for each mode
for mode in ordered writeback journal; do
  echo "=== Testing $mode mode ==="
  fio --section=seqwrite \
      --section=randwrite \
      --section=fsync-test \
      --mode=$mode journal-test.fio
done
```

Analysis

```
# Check fragmentation
for mode in ordered writeback journal; do
 echo "=== $mode fragmentation ==="
 filefrag /mnt/${mode}/seqwrite.*
done

# Clean up
umount /mnt/{ordered,writeback,journal}
```

Hands-On Lab — Quota Debugging

Lab Exercise: Trace Quota Updates

Setup Quota

```
# Create test filesystem with quota
dd if=/dev/zero of=/tmp/quota_test.img \
  bs=1M count=512
mkfs.ext4 -F -O quota /tmp/quota_test.img

# Mount with quota
mkdir /mnt/quota
mount -o loop,usrquota,grpquota \
 /tmp/quota_test.img /mnt/quota

# Initialize quota files
quotacheck -cug /mnt/quota
quotaon /mnt/quota

# Set limits
setquota -u testuser \
 100M 120M 1000 1200 /mnt/quota
```

Tracing

```
# Terminal 1: Trace quota events
trace-cmd record -e quota* sleep 30

# Terminal 2: Generate quota activity
sudo -u testuser bash -c "
 cd /mnt/quota
 for i in {1..200}; do
 dd if=/dev/zero of=file\$i \
 bs=1M count=1 2>/dev/null
 done
"

# Analyze trace
trace-cmd report | \
 grep -E "alloc_space|exceeded"

# Debug specific issues
cat /sys/kernel/debug/tracing/trace | \
 grep -B5 -A5 "EDQUOT"
```

Module 4 Summary

Metadata Operations Key Takeaways

Allocation Strategies

- **Inode allocation**: Near parent directory for locality
- **Block allocation**: mballoc with preallocation
- **Delayed allocation**: Better contiguous allocation
- **Orlov allocator**: Spread directories evenly

Directory Operations

- **Linear**: Small directories (simple)
- **HTree**: Large directories (O(log n) lookup)
- **Hash functions**: TEA, half-MD4, legacy

Extended Attributes

- **Inline**: In inode extra space (fast)
- **External**: Separate block (larger)
- **Namespace separation**: security, system, trusted, user

Quota Management

- **Three types**: User, group, project
- **Update path**: On allocation/free
- **Limits**: Hard and soft with grace periods
- **Debugging**: tracepoints, quotacheck

Metadata Recovery

- **Checksums**: Detect corruption
- **Backup groups**: Redundant descriptors
- **Journal replay**: Transaction-based recovery
- **e2fsck**: Offline repair

Q&A and Discussion

Open Topics

Discussion Questions

1. **Journal mode selection**: When would you choose each mode in production?
2. **Barrier trade-offs**: Is disabling barriers ever safe?
3. **Delalloc risks**: How to monitor and prevent ENOSPC issues?
4. **HTree limitations**: When does it break down?

Common Issues

- **Journal too small**: Frequent checkpoints hurt performance
- **Quota delays**: Async writes can delay limit enforcement
- **xattr fragmentation**: Many small xattrs cause overhead
- **Directory scaling**: Millions of files need HTree tuning

Best Practices

- Monitor journal usage: `dumpe2fs /dev/sda1 | grep -i journal`
- Tune commit interval for workload
- Use project quotas for container isolation
- Regular fstrim for SSD devices

Preview of Day 3

Coming Up Tomorrow

Module 5: Extent-Based Allocation

- Extent tree operations
- Delayed allocation deep dive
- Fragmentation avoidance
- Low-memory handling

Module 6: I/O Path & Memory Integration

- VFS integration
- Page cache mechanics
- Direct vs buffered I/O
- Folio interface
- Zoned device support

Hands-On Labs

- blktrace I/O analysis
- Page cache experiments
- Memory pressure testing

Preparation: Review extent concepts from Day 1 · Install blktrace and perf · Prepare SSD/NVMe for testing

Additional Resources

References and Further Reading

Kernel Documentation

- `Documentation/filesystems/ext4/journal.rst`
- `Documentation/admin-guide/quota.rst`
- `Documentation/filesystems/ext4/directory.rst`

Source Files to Study

- `fs/ext4/inode.c` — Inode operations
- `fs/ext4/ialloc.c` — Inode allocation
- `fs/ext4/mballoc.c` — Multiblock allocator
- `fs/jbd2/transaction.c` — Journal transactions
- `fs/ext4/xattr.c` — Extended attributes
- `fs/ext4/quota.c` — Quota management

Tools Reference

```
# Journal debugging
debugfs -R "logdump" /dev/sda1
dumpe2fs /dev/sda1 | grep -A10 "Journal"

# Quota tools
man quotactl
man setquota
man repquota

# Performance analysis
perf list | grep ext4
perf list | grep jbd2
```

End of Day 2 Presentation

Questions?

Instructor

Chandrashekar Babu

https://www.chandrashekar.info/

https://www.slashprog.com/

training@chandrashekar.info

Day 3 Preview

Extent-Based Allocation, I/O Path & Memory Integration, and Hands-On Labs with blktrace and page cache experiments.